Relational Model, Relational Algebra & Advanced SQL

PBEOT504: Database Management System (S5)

Bachelor of Technology (B.Tech) — 2024 Scheme

Complete Technical Notes, ER-to-Relational Synthesis & SQL Query Suite

Prepared from Course Materials, Lecture Slides & KTU Curriculum

Module 2 Complete Syllabus Coverage:

- Structure of Relational Databases: Keys, Integrity Constraints & Domains
- 7-Step Algorithm for Synthesizing ER Diagrams to Relational Schema
- Relational Algebra: Selection, Projection, Joins, Division & Complex Queries
- Structured Query Language (SQL): DDL, DML, Nested Queries, Aggregations
- Advanced SQL: Views, Triggers, Assertions, CTEs & JDBC Host Connectivity
- Comprehensive Problem Suite & KTU Model Examination Questions

Compiled August 17, 2026

Contents

1 Structure of Relational Databases and Integrity Constraints 2
1.1 Fundamental Concepts of the Relational Model . . . 2
1.2 Relational Keys: Taxonomy and Definitions . . . 2
1.3 Relational Integrity Constraints . . . 3
2 Synthesizing ER Diagrams to Relational Schema 4
3 Relational Algebra: The Formal Query Language 6
3.1 Unary Relational Operators . . . 6
3.2 Set-Theoretic Relational Operators . . . 6
3.3 Binary Relational Join Operations . . . 6
4 Structured Query Language (SQL) 8
4.1 SQL Data Definition Language (DDL) . . . 8
4.2 SQL Data Manipulation Language (DML) . . . 8
4.3 Advanced SQL Retrieval: Joins, Nested Queries, and Aggregations . . . 9
4.4 Views, Assertions, Triggers, and Recursive Queries . . . 9
5 Comprehensive Relational & SQL Problem Suite 11
6 Quick Revision Sheet & Model Examination Questions 13
6.1 Master Summary Table: Relational Algebra Operators vs. SQL Constructs . . . 13
6.2 Model Examination Questions (KTU S5 Pattern) . . . 13

1 Structure of Relational Databases and Integrity Constraints

Syllabus Requirement: Structure of Relational Databases - Integrity Constraints, Synthesizing ER diagram to relational schema

1.1 Fundamental Concepts of the Relational Model

Introduced by Dr. Edgar F. Codd in 1970 at IBM, the **Relational Model** represents a database as a collection of mathematically formal two-dimensional tables called **Relations**.

Formal Relational Term	Informal / Practical Term	Precise Mathematical Definition
Relation	Table / File	A subset of the Cartesian product of attribute domains: $R \subseteq \text{dom}(A_1) \times \cdots \times \text{dom}(A_n)$
Tuple	Row / Record	An ordered list of n values $\langle v_1, v_2, \ldots, v_n \rangle$ where $v_i \in \text{dom}(A_i)$
Attribute	Column Header / Field	A named role defining the interpretation of values from domain $\text{dom}(A)$
Domain	Data Type / Range	Set of atomic, indivisible values permissible for an attribute
Degree (n)	Arity / Column Count	The total number of attributes in the relation schema $R(A_1, \ldots, A_n)$
Cardinality (m)	Row Count / Record Count	The total number of tuples currently present in relation state $r(R)$
Relation Schema	Table Structure	Denoted by $R(A_1, A_2, \ldots, A_n)$; static description (intension)
Relation State	Table Data / Instance	The set of tuples $r(R) = \{t_1, t_2, \ldots, t_m\}$ at a given time (extension)

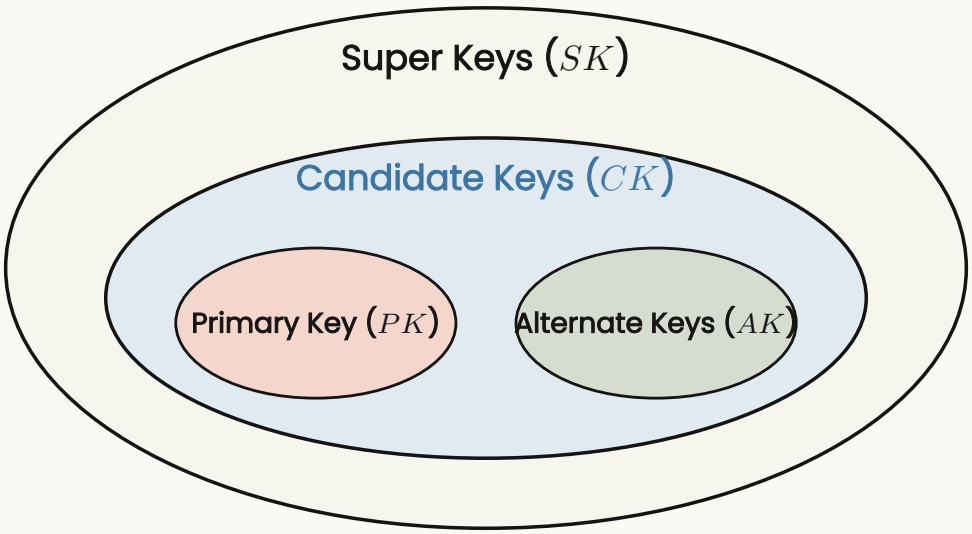
Characteristics of Relational Databases

1. **No Duplicate Tuples:** By mathematical definition, a relation is a set of distinct tuples. Duplicate rows are strictly disallowed.
2. **Ordering of Tuples is Irrelevant:** Tuples have no inherent physical sequence inside a relation.
3. **Ordering of Attributes is Irrelevant:** Attributes are identified by unique column names, not position.
4. **Atomicity of Values (First Normal Form - 1NF):** Every attribute value in a tuple must be an indivisible atomic value. Multivalued or composite attributes are forbidden in normalized relational tables.
5. **The NULL Value Concept:** Used to represent values that are unknown, missing, or inapplicable.

1.2 Relational Keys: Taxonomy and Definitions

Keys are fundamental mathematical constructs that ensure tuple uniqueness and establish relationships between tables.

Key Definitions: **1. Super Key (SK):** A set of one or more attributes that collectively uniquely identifies a tuple within a relation. For any two distinct tuples $t_1, t_2 \in r(R)$, $t_1[SK] \neq t_2[SK]$. A super key may contain extraneous attributes.
2. Candidate Key (CK): A **minimal Super Key**. A super key K is a candidate key if removing any attribute from K causes it to lose the uniqueness property. A relation may possess multiple candidate keys.
3. Primary Key (PK): The specific candidate key chosen by the database designer to identify tuples uniquely throughout the database. Its values **can never be NULL**.
4. Alternate Key (AK): Any candidate key that was **not chosen** as the primary key ($AK = CK \setminus \{PK\}$).
5. Foreign Key (FK): A set of attributes in a referencing relation R_1 whose values are required to match the primary key PK of a referenced relation R_2, or else be NULL.



1.3 Relational Integrity Constraints

Integrity constraints are conditions that must hold true on all valid database states.

1. **Domain Constraints:** Every value $t[A]$ must be an atomic value belonging to the domain $\text{dom}(A)$ (data type, length, range restrictions, `CHECK` constraints).
2. **Entity Integrity Constraint:** The primary key attributes of a relation schema R **cannot accept NULL values** ($t[PK] \neq \text{NULL}$ for any tuple t). Rationale: NULL values cannot uniquely identify individual tuples.
3. **Key / Uniqueness Constraint:** No two tuples in a relation can have identical values for any candidate key ($t_1[CK] \neq t_2[CK]$).
4. **Referential Integrity Constraint:** A foreign key FK in relation R_1 referencing relation $R_2(PK)$ must satisfy:

$$t_1[FK] = t_2[PK] \quad \text{for some } t_2 \in R_2, \quad \text{or} \quad t_1[FK] = \text{NULL} \tag{1}$$

Handling Violations of Referential Integrity

When an `UPDATE` or `DELETE` occurs on a referenced table, the DBMS can enforce:

- `ON DELETE RESTRICT` (or `NO ACTION`): Rejects the delete operation if child records exist.
- `ON DELETE CASCADE`: Automatically deletes all dependent child tuples.
- `ON DELETE SET NULL`: Automatically sets the foreign key of child tuples to NULL.
- `ON DELETE SET DEFAULT`: Sets the child foreign key to a specified default value.

2 Synthesizing ER Diagrams to Relational Schema

The logical database design process converts a conceptual ER diagram into a normalized set of relational tables through a rigorous **7-Step Mapping Algorithm**.

The 7-Step ER-to-Relational Synthesis Algorithm

Step 1: Mapping Strong (Regular) Entity Types

For each strong entity type E, create a relation R containing all simple attributes of E.

- **Composite Attributes:** Include only their simple component sub-attributes (e.g., `Name` → `Fname`, `Minit`, `Lname`).
- **Primary Key:** Primary key of R = primary key of entity E.

Step 2: Mapping Weak Entity Types

For each weak entity type W with identifying owner entity E:

- Create a relation R_W containing all simple attributes of W.
- Add the **Primary Key of** E as a **Foreign Key** in R_W.
- **Primary Key of** R_W**:** The composite key $\{\text{Primary Key of } E\} \cup \{\text{Partial Key of } W\}$.

Step 3: Mapping Multivalued Attributes

For each multivalued attribute A in entity E:

- Create a separate relation R_A.
- Include attribute A and the **Primary Key of** E (as a Foreign Key).
- **Primary Key of** R_A**:** $\{\text{Primary Key of } E, A\}$.

Step 4: Mapping Binary 1:1 Relationships

For a binary $1:1$ relationship between entities S and T, choose one of three approaches:

1. **Foreign Key Approach (Standard / Preferred):** Add the primary key of S as a foreign key in T (prefer putting FK in the entity with **total participation** to prevent NULLs). Include any relationship attributes in T.
2. **Merged Relation Approach:** If **both** entities participate totally, merge S and T into a single consolidated relation.
3. **Cross-Reference / Junction Relation:** Create a third relation $R(PK_S, PK_T)$ (used if both sides have partial participation).

Step 5: Mapping Binary 1:N (and N:1) Relationships

For a $1:N$ relationship between entity S (1-side) and entity T (N-side):

- Identify the relation on the N**-side** (T).
- Add the **Primary Key of the** 1**-side** (S) as a **Foreign Key** inside relation T.
- Include any relationship attributes inside T.

Step 6: Mapping Binary M:N Relationships

For an $M:N$ relationship between entities S and T:

- Create a separate **Junction / Cross-Reference Relation** R_{ST}.
- Include the primary key of S (FK_S) and primary key of T (FK_T) as foreign keys.
- **Primary Key of** R_{ST}**:** The composite key $\{FK_S, FK_T\}$.
- Include all relationship attributes inside R_{ST}.

Step 7: Mapping Recursive and N-ary (Ternary) Relationships

- **Recursive** $1:N$ **Relationship:** Add a foreign key in the same relation referencing its own primary key (e.g., `EMPLOYEE(SSN, Name, Supervisor_SSN)`).

- **Ternary / N-ary Relationship:** Create a separate relation R containing the foreign keys of all N participating entities plus any relationship attributes. Primary key is the composite of all participating primary keys.

3 Relational Algebra: The Formal Query Language

Syllabus Requirement: Relational Algebra - Select, Project, Rename, Set operations, Cartesian Product, Join operations (Inner, Theta, Equi, Natural, Outer Joins), Division operator, Relational Algebra queries

3.1 Unary Relational Operators

Relational algebra is a procedural query language consisting of operations that take one or two relations as input and produce a new relation as output.

1. The Select Operation (σ)

The **Select** operator (σ) filters rows (tuples) that satisfy a boolean selection condition:

$$\sigma_{\langle \text{condition} \rangle}(R) = \{t \mid t \in R \text{ and condition}(t) = \text{TRUE}\} \tag{2}$$

- *Degree:* Same as degree of R ($\deg(\sigma(R)) = \deg(R)$).
- *Cardinality:* $|\sigma(R)| \leq |R|$.
- *Example:* $\sigma_{\text{Age}>20 \wedge \text{Branch}='\text{EEE}'}(\text{STUDENT})$.

2. The Project Operation (Π)

The **Project** operator (Π) selects specified columns (attributes) and discards the rest:

$$\Pi_{A_1, A_2, \ldots, A_k}(R) \tag{3}$$

- *Duplicate Elimination:* By definition of a mathematical set, Π **automatically eliminates duplicate tuples** from the resulting relation.
- *Example:* $\Pi_{\text{RollNo, Name}}(\sigma_{\text{Age}>20}(\text{STUDENT}))$.

3. The Rename Operation (ρ)

Used to rename relations or attributes:

$$\rho_{S(B_1, B_2, \ldots, B_n)}(R) \quad \text{or} \quad \rho_S(R) \tag{4}$$

3.2 Set-Theoretic Relational Operators

Two relations $R(A_1, \ldots, A_n)$ and $S(B_1, \ldots, B_n)$ are **Union-Compatible** (type-compatible) if:

1. They have the same degree ($\deg(R) = \deg(S) = n$).
2. The domains of corresponding attributes are identical: $\text{dom}(A_i) = \text{dom}(B_i)$ for all $1 \leq i \leq n$.

Set Operations:

- **Union ($R \cup S$):** Set of all tuples that appear in R, in S, or in both.
- **Intersection ($R \cap S$):** Set of tuples appearing in both R and S ($R \cap S = R - (R - S)$).
- **Set Difference ($R - S$):** Set of tuples in R that do not appear in S.
- **Cartesian Product ($R \times S$):** Combines every tuple in R with every tuple in S.

$$\deg(R \times S) = \deg(R) + \deg(S), \qquad |R \times S| = |R| \times |S| \tag{5}$$

3.3 Binary Relational Join Operations

1. Theta Join ($\bowtie_\theta$)

Combines related tuples from two relations satisfying a condition $\theta \in \{=, <, \leq, >, \geq, \neq\}$:

$$R \bowtie_\theta S = \sigma_\theta(R \times S) \tag{6}$$

2. Equi-Join

A theta join where the comparison operator is strictly equality ($=$). Duplicate attribute columns remain in the output schema.

3. Natural Join ($R * S$ or $R \bowtie S$)

An Equi-Join performed over all common attribute names in R and S, followed by **automatic projection to remove duplicate attribute columns**:

$$R \bowtie S = \Pi_{\text{Attributes}}(\sigma_{R.A_1=S.A_1 \wedge \cdots \wedge R.A_k=S.A_k}(R \times S)) \tag{7}$$

4. Outer Joins (Preserving Dangling Tuples)

- **Left Outer Join ($R \,]\bowtie S$):** Keeps all tuples from left relation R. If no match exists in S, the S-attributes are filled with `NULL`.
- **Right Outer Join ($R \bowtie[\, S$):** Keeps all tuples from right relation S, padding unmatched R-attributes with `NULL`.
- **Full Outer Join ($R \,]\bowtie[\, S$):** Keeps all tuples from both relations, padding unmatched fields with `NULL`.

5. Division Operator ($\div$)

The **Division Operator** ($R \div S$) answers queries involving universal quantification ("Find entities associated with **ALL** items in S").

$$R \div S = \Pi_{A-B}(R) - \Pi_{A-B}\left((\Pi_{A-B}(R) \times S) - R\right) \tag{8}$$

Condition: Attributes of S must be a proper subset of attributes of R ($B \subset A$). The result contains attributes $(A - B)$.

4 Structured Query Language (SQL)

Syllabus Requirement: Structured Query Language (SQL), Data Definition Language (DDL), Table definitions and operations – CREATE, DROP, ALTER, INSERT, DELETE, UPDATE. SQL DML - SQL queries on single and multiple tables, Nested queries, Aggregation and grouping, Views, assertions, Triggers, SQL data types, Recursive queries, Accessing SQL from a Programming Language

4.1 SQL Data Definition Language (DDL)

1. Table Creation with Comprehensive Constraints

```sql
CREATE TABLE DEPARTMENT (
    DNumber     INT PRIMARY KEY,
    DName       VARCHAR(50) NOT NULL UNIQUE,
    Manager_SSN CHAR(9),
    Mgr_Start_Date DATE
);

CREATE TABLE EMPLOYEE (
    SSN         CHAR(9) PRIMARY KEY,
    FName       VARCHAR(30) NOT NULL,
    LName       VARCHAR(30) NOT NULL,
    Salary      DECIMAL(10,2) CHECK (Salary >= 10000.00),
    DNo         INT NOT NULL,
    Supervisor_SSN CHAR(9),
    FOREIGN KEY (DNo) REFERENCES DEPARTMENT(DNumber)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (Supervisor_SSN) REFERENCES EMPLOYEE(SSN)
        ON DELETE SET NULL ON UPDATE CASCADE
);
```

2. Schema Modification and Table Deletion

```sql
-- Add new column
ALTER TABLE EMPLOYEE ADD Email VARCHAR(100);

-- Modify column data type
ALTER TABLE EMPLOYEE MODIFY Salary DECIMAL(12,2);

-- Drop table and related constraints
DROP TABLE EMPLOYEE CASCADE;

-- Delete all rows while preserving table structure
TRUNCATE TABLE EMPLOYEE;
```

4.2 SQL Data Manipulation Language (DML)

```sql
-- Insert single row
INSERT INTO EMPLOYEE (SSN, FName, LName, Salary, DNo)
VALUES ('123456789', 'John', 'Smith', 55000.00, 5);

-- Update tuples conditionally
UPDATE EMPLOYEE
SET Salary = Salary * 1.10
WHERE DNo = 5;
```

```sql
-- Delete tuples conditionally
DELETE FROM EMPLOYEE
WHERE Salary < 20000.00;
```

4.3 Advanced SQL Retrieval: Joins, Nested Queries, and Aggregations

1. The SELECT-FROM-WHERE Framework and Multi-Table Joins

```sql
-- Find Employee Names and their Department Names (Explicit Inner Join)
SELECT E.FName, E.LName, D.DName
FROM EMPLOYEE E
JOIN DEPARTMENT D ON E.DNo = D.DNumber
WHERE E.Salary > 40000.00;
```

2. Aggregate Functions, GROUP BY, and HAVING Clause

- **Aggregate Functions:** `COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`.
- **WHERE vs. HAVING:** `WHERE` filters individual rows *before* grouping; `HAVING` filters aggregated groups *after* grouping.

```sql
-- Find Departments having more than 5 employees with average salary > 50000
SELECT DNo, COUNT(*) AS Total_Employees, AVG(Salary) AS Avg_Salary
FROM EMPLOYEE
WHERE Salary >= 20000.00
GROUP BY DNo
HAVING COUNT(*) > 5 AND AVG(Salary) > 50000.00;
```

3. Nested Subqueries: Non-Correlated vs. Correlated

```sql
-- Non-Correlated Subquery using IN
SELECT FName, LName FROM EMPLOYEE
WHERE DNo IN (SELECT DNumber FROM DEPARTMENT WHERE DName = 'Research');

-- Correlated Subquery using EXISTS (Employees who work on all department
   projects)
SELECT E.FName, E.LName FROM EMPLOYEE E
WHERE NOT EXISTS (
    SELECT P.PNumber FROM PROJECT P WHERE P.DNum = E.DNo
    EXCEPT
    SELECT W.PNo FROM WORKS_ON W WHERE W.ESSN = E.SSN
);
```

4.4 Views, Assertions, Triggers, and Recursive Queries

1. Views: Virtual Tables

A **View** is a dynamic virtual table defined by an underlying SQL query. It does not store physical data independently:

```sql
CREATE VIEW EEE_DEPT_EMPS AS
SELECT E.SSN, E.FName, E.LName, E.Salary
FROM EMPLOYEE E
WHERE E.DNo = (SELECT DNumber FROM DEPARTMENT WHERE DName = 'EEE');
```

Benefits: Security abstraction, simplified complex queries, logical data independence.

2. Database Triggers (Event-Condition-Action / ECA Model)

```
CREATE TRIGGER check_min_salary
BEFORE INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
    IF NEW.Salary < 15000.00 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be less than minimum wage 15000!';
    END IF;
END;
```

3. Recursive Queries using Common Table Expressions (CTE)

```
-- Find the complete management hierarchy under CEO
WITH RECURSIVE OrgHierarchy AS (
    -- Anchor Member
    SELECT SSN, FName, Supervisor_SSN, 1 AS Level
    FROM EMPLOYEE WHERE Supervisor_SSN IS NULL
    UNION ALL
    -- Recursive Member
    SELECT E.SSN, E.FName, E.Supervisor_SSN, O.Level + 1
    FROM EMPLOYEE E
    JOIN OrgHierarchy O ON E.Supervisor_SSN = O.SSN
)
SELECT * FROM OrgHierarchy;
```

4. Accessing SQL from Programming Languages: JDBC Architecture

Java DataBase Connectivity (**JDBC**) is a standard Java API that enables Java programs to execute SQL statements.

1. `DriverManager` : Manages database drivers and establishes database connections.
2. `Connection` : Represents an active session with the specific database.
3. `PreparedStatement` : Pre-compiled SQL statements preventing SQL Injection attacks.
4. `ResultSet` : Cursor-based data structure used to iterate through rows retrieved by a query.

5 Comprehensive Relational & SQL Problem Suite

Problem 1: Synthesizing Complex ER Diagram to Relational Schema

Convert the following University ER scenario into a normalized Relational Database Schema:

- `PROFESSOR` (EmpNo, Name, Designation)
- `DEPARTMENT` (Dept_ID, Dept_Name). A department offers multiple courses. A course is offered by exactly 1 department.
- `COURSE` (CourseID, Course_Name, Credits). Has multivalued attribute `Prerequisites`.
- `STUDENT` (RollNo, Name, Address(Street, City, Pincode), Phone_Numbers).
- Relationships: `TEACHES` (M : N between `PROFESSOR` and `COURSE` with attribute `Semester`), `ENROLLS` (M : N between `STUDENT` and `COURSE` with attribute `Grade`).

Step-by-Step Solution

Applying the 7-Step Synthesis Algorithm:

1. **`DEPARTMENT`** (Dept_ID, Dept_Name)
2. **`PROFESSOR`** (EmpNo, Name, Designation, $\text{Dept_ID}_{\text{FK}}$)
3. **`COURSE`** (CourseID, Course_Name, Credits, $\text{Dept_ID}_{\text{FK}}$)
4. **`COURSE_PREREQUISITES`** ($\text{CourseID}_{\text{FK}}$, $\text{Prereq_CourseID}_{\text{FK}}$)
5. **`STUDENT`** (RollNo, Name, Street, City, Pincode)
6. **`STUDENT_PHONES`** ($\text{RollNo}_{\text{FK}}$, Phone_Number)
7. **`TEACHES`** (EmpNo_{FK}, $\text{CourseID}_{\text{FK}}$, Semester)
8. **`ENROLLS`** ($\text{RollNo}_{\text{FK}}$, $\text{CourseID}_{\text{FK}}$, Grade)

Problem 2: Relational Algebra Query Suite on University Schema

Given the schema:

- STUDENT(RollNo, Name, Age, Gender)
- COURSE(CourseID, CName, Credits)
- ENROLLS(RollNo, CourseID, Grade)
- PROFESSOR(EmpNo, PName, Phone)
- TEACHES(EmpNo, CourseID)

Write relational algebra expressions for:

1. Find the Credits of course with name `'DBMS'`.
2. Find RollNo, Name, and Gender of students enrolled in `'Chemistry'`.
3. Find Names and Phone numbers of professors who taught at least one course.
4. Find RollNo and Names of students who enrolled in **ALL** courses.

Step-by-Step Solution

1. $\Pi_{\text{Credits}}\left(\sigma_{\text{CName}='\text{DBMS}'}(\text{COURSE})\right)$
2. $\Pi_{\text{RollNo, Name, Gender}}\left(\text{STUDENT} \bowtie \text{ENROLLS} \bowtie \sigma_{\text{CName}='\text{Chemistry}'}(\text{COURSE})\right)$
3. $\Pi_{\text{PName, Phone}}\left(\text{PROFESSOR} \bowtie \text{TEACHES}\right)$
4. $\Pi_{\text{RollNo, Name}}\left(\text{STUDENT} \bowtie \left(\Pi_{\text{RollNo, CourseID}}(\text{ENROLLS}) \div \Pi_{\text{CourseID}}(\text{COURSE})\right)\right)$

Problem 3: Advanced SQL Queries with Aggregations and Subqueries

Using the University schema, write SQL queries for:

1. Find the names of students who have enrolled in courses with more than 3 credits and received grade 'A'.
2. Find the CourseID and CName of courses that have more than 30 students enrolled.
3. Find professors who have never taught any course.
4. For each department, display Dept_Name and average salary of professors in that department.

Step-by-Step Solution

```sql
-- Query 1
SELECT S.Name
FROM STUDENT S
JOIN ENROLLS E ON S.RollNo = E.RollNo
JOIN COURSE C ON E.CourseID = C.CourseID
WHERE C.Credits > 3 AND E.Grade = 'A';

-- Query 2
SELECT C.CourseID, C.CName, COUNT(E.RollNo) AS Student_Count
FROM COURSE C
JOIN ENROLLS E ON C.CourseID = E.CourseID
GROUP BY C.CourseID, C.CName
HAVING COUNT(E.RollNo) > 30;

-- Query 3
SELECT PName FROM PROFESSOR
WHERE EmpNo NOT IN (SELECT EmpNo FROM TEACHES);

-- Query 4
SELECT D.Dept_Name, AVG(P.Salary) AS Avg_Salary
FROM DEPARTMENT D
JOIN PROFESSOR P ON D.Dept_ID = P.Dept_ID
GROUP BY D.Dept_ID, D.Dept_Name;
```

6 Quick Revision Sheet & Model Examination Questions

6.1 Master Summary Table: Relational Algebra Operators vs. SQL Constructs

Relational Operator	Algebraic Notation	Equivalent SQL Clause	Core Mathematical Behavior
Selection	$\sigma_{\langle \text{cond} \rangle}(R)$	`SELECT * WHERE cond`	Horizontal tuple filter
Projection	$\Pi_{\langle \text{attrs} \rangle}(R)$	`SELECT DISTINCT attrs`	Vertical column extraction
Cartesian Product	$R \times S$	`CROSS JOIN`	Combines every row in R with S
Theta Join	$R \bowtie_\theta S$	`JOIN ... ON cond`	Cross product followed by σ_θ
Natural Join	$R \bowtie S$	`NATURAL JOIN`	Equi-join on common attribute names
Left Outer Join	$R \,]\bowtie S$	`LEFT OUTER JOIN`	Preserves all left tuples; pads with NULL
Set Union	$R \cup S$	`UNION`	Requires union-compatible schemas
Set Difference	$R - S$	`EXCEPT` / `MINUS`	Tuples in R not in S
Division Operator	$R \div S$	`NOT EXISTS ... EXCE`	Universal quantification (”for ALL in S”)

6.2 Model Examination Questions (KTU S5 Pattern)

University Examination Pattern (PBEOT504 Scheme)

The End Semester Examination consists of **Part A** (8 compulsory questions of 2 marks each, 2 from each module) and **Part B** (Module choice questions carrying 6 marks each).

Part A: Short Answer Questions (2 Marks Each)

1. **Q1:** Differentiate between a *Candidate Key* and a *Super Key*. Can a primary key contain a NULL value?
2. **Q2:** State the *Entity Integrity Constraint* and *Referential Integrity Constraint* with mathematical formulations.
3. **Q3:** Differentiate between *Equi-Join* and *Natural Join* in Relational Algebra.
4. **Q4:** What is the purpose of the *Division Operator* ($\div$) in Relational Algebra? Give an example.
5. **Q5:** Explain the difference between `WHERE` and `HAVING` clauses in SQL.
6. **Q6:** What is a *View* in SQL? State two advantages of using views.

Part B: Long Answer / Analytical Questions (6 Marks Each)

1. **Q7 (a):** State and explain the 7-step mapping algorithm to synthesize an ER diagram into a relational schema. (4 Marks)
   Q7 (b): How are weak entities and multivalued attributes mapped into relational tables? (2 Marks)

2. **Q8 (a):** Consider the schema: SAILOR(sid, sname, rating, age), BOAT(bid, bname, color), and RESERVES(sid, bid, day). Write Relational Algebra queries to:
   (i) Find names of sailors who reserved a red boat.
   (ii) Find names of sailors who reserved **all** boats. (6 Marks)
3. **Q9 (a):** Explain the architecture of JDBC. Write the key Java code steps to establish a connection and execute a `PreparedStatement` query. (6 Marks)

— End of Module 2 Study Notes —